Rhea Madhogarhia (my brother is also in the class)

Project 1: Representation Learning

Objectives

Your goal in this project is to get comfortable in implementing k-means and PCA

Deliverable

- Project report/writeup: A `project1_report_lastname.pdf` file generated from the below document.

  The project report should be generated by, following completion of the assignment, printing the resulting document to a pdf file and submitting that file. The document should additionally include a brief justification of your solution at a high-level, e.g., using any relevant explanations, equations, or pictures that help to explain your solution. You should **answer each question in-line**, and all questions we ask you to answer must be present.. You should also **describe what your code does**, e.g. using a couple of sentences per function to describe your code structure, in a section following the code itself. The objective is to make the report self-contained for grading, while ensuring that you wrote the code yourself.

    - To be specific, this means your report will include (but not be limited to):
        1. Any code snippets you implemented
        2. Plots affected by your implementation, e.g., scatter plots with decision boundaries or loss curves.
        3. Output results of the questions.
        4. Please do NOT include debugging messages.

Logistics

- You complete and submit the project report/writeup individually. While you may discuss the core concepts, the logic, and the assigned questions with others, the source code and project report needs to be

written independently.
- Source code should be left in-line below. Failure to provide source code will lead to a deduction of points from your total.

Task 1: K-Means (50 points)

K-Means is an unsupervised ML algorithm that segments data into groups based on their similarities. In this part, we will implement K-Means with different seeding algorithms from scratch (using only numpy).

Data Loader

We will use 2-dimensional data in this section. Specifically, the dataset are 2D points $[(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)]$ that form 5 distinct clusters.

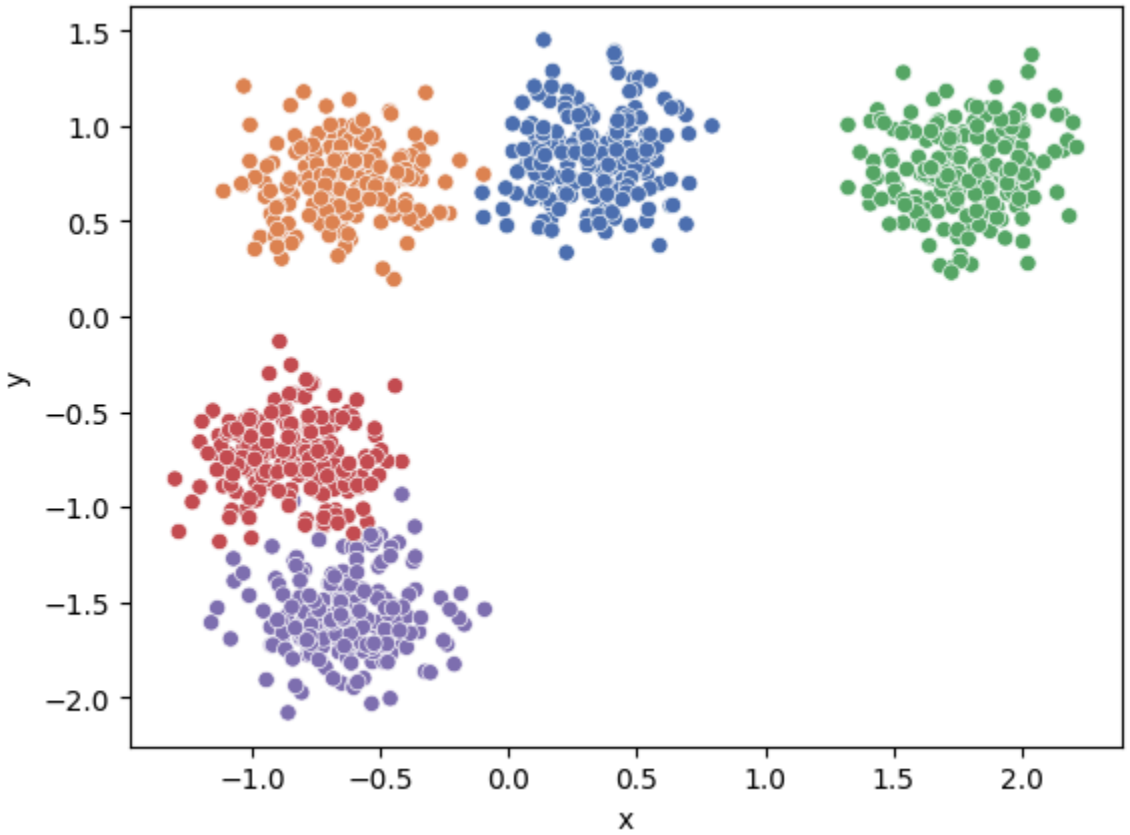
In [19]:
```
# install dependencies.
#sorry had to change this to pip3 since thats what my environment uses
!pip3 install seaborn scikit-learn numpy pandas matplotlib
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: seaborn in /Users/rheamadhogarhia/Library/Pytho
n/3.9/lib/python/site-packages (0.13.2)
Requirement already satisfied: scikit-learn in /Users/rheamadhogarhia/Library/P
ython/3.9/lib/python/site-packages (1.5.2)
Requirement already satisfied: numpy in /Users/rheamadhogarhia/Library/Python/
3.9/lib/python/site-packages (2.0.2)
Requirement already satisfied: pandas in /Users/rheamadhogarhia/Library/Python/
3.9/lib/python/site-packages (2.2.3)
Requirement already satisfied: matplotlib in /Library/Python/3.9/site-packages
(3.9.4)
Requirement already satisfied: scipy>=1.6.0 in /Users/rheamadhogarhia/Library/P
ython/3.9/lib/python/site-packages (from scikit-learn) (1.13.1)
Requirement already satisfied: joblib>=1.2.0 in /Users/rheamadhogarhia/Library/
Python/3.9/lib/python/site-packages (from scikit-learn) (1.5.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /Users/rheamadhogarhia/L
ibrary/Python/3.9/lib/python/site-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: tzdata>=2022.7 in /Users/rheamadhogarhia/Librar
y/Python/3.9/lib/python/site-packages (from pandas) (2025.2)
Requirement already satisfied: pytz>=2020.1 in /Users/rheamadhogarhia/Library/P
ython/3.9/lib/python/site-packages (from pandas) (2025.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /Users/rheamadhogarhi
a/Library/Python/3.9/lib/python/site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: contourpy>=1.0.1 in /Library/Python/3.9/site-pac
kages (from matplotlib) (1.3.0)
Requirement already satisfied: fonttools>=4.22.0 in /Library/Python/3.9/site-pa
ckages (from matplotlib) (4.60.1)
Requirement already satisfied: cycler>=0.10 in /Library/Python/3.9/site-package
s (from matplotlib) (0.12.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /Library/Python/3.9/site-pa
ckages (from matplotlib) (1.4.7)
Requirement already satisfied: pyparsing>=2.3.1 in /Library/Python/3.9/site-pac
kages (from matplotlib) (3.2.5)
Requirement already satisfied: packaging>=20.0 in /Users/rheamadhogarhia/Librar
y/Python/3.9/lib/python/site-packages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in /Library/Python/3.9/site-packages
(from matplotlib) (11.3.0)
Requirement already satisfied: importlib-resources>=3.2.0 in /Library/Python/
3.9/site-packages (from matplotlib) (6.5.2)
Requirement already satisfied: zipp>=3.1.0 in /Users/rheamadhogarhia/Library/Py
thon/3.9/lib/python/site-packages (from importlib-resources>=3.2.0->matplotlib)
(3.23.0)
Requirement already satisfied: six>=1.5 in /Library/Developer/CommandLineTools/
Library/Frameworks/Python3.framework/Versions/3.9/lib/python3.9/site-packages
(from python-dateutil>=2.8.2->pandas) (1.15.0)
WARNING: You are using pip version 21.2.4; however, version 25.3 is available.
You should consider upgrading via the '/Library/Developer/CommandLineTools/usr/
bin/python3 -m pip install --upgrade pip' command.
```

Prepare and plot the training set.

In [20]:
```
import seaborn as sns
from sklearn.datasets import make_blobs
```

```python
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler

num_samples = 1000
num_centers = 5


X_train, true_labels = make_blobs(
    n_samples=num_samples,
    centers=num_centers,
    random_state=80,
)
X_train = StandardScaler().fit_transform(X_train)
sns.scatterplot(
    x=[X[0] for X in X_train],
    y=[X[1] for X in X_train],
    hue=true_labels,
    palette="deep",
    legend=None,
)

plt.xlabel("x")
plt.ylabel("y")
plt.show()
```



K-Means Implementation

Now we are implementing the algorithm.

First, we define "similarity" between points. For our dataset, we will be using Euclidean distances.

1. **Implement a helper function** that has two inputs: `dataset`, `cur_point`, and returns the Euclidean distance between `cur_point` and each point in the dataset.

In [ ]:
```
import numpy as np


def euclidean_dist(cur_point, dataset):
    '''
    cur_point has dimensions (m,), dataset has dimensions (n, m), and output w
    '''
    dists = [] #not sure if I was supposed to keep this init as None
    # Your Code Starts Here
    for point in dataset:
        #just a basic euclidean distance formula
        #sqrt of sum of squared differences
        euc_dist = np.sqrt(np.sum((cur_point - point) ** 2))
        dists.append(euc_dist)
    # End of Your Code

    return dists
```

Now, we implement the actual K-Means algorithm. We first initialize K-Means with two parameters: number of clusters, and max number of iteractions, which you will use in the init function.

1. **Implent the initialization functions** to accommodate four different seeding strategies: first k, random, max distance, and k-means++.
2. **Implement the fit function** which takes the dataset you created earlier as input.

Finally, we can assign data points to the closest centroid in the provided predict function.

In [ ]:
```
from enum import Enum
import random
from numpy.random import uniform

class InitializationMethod(Enum):
    FirstK = 1
    Random = 2
```

```
    MaxDistance = 3
    KMeansPlusPlus = 4


class KMeans:

    def __init__(self, n_clusters: int = 5, max_iter: int = 1000):
        self.n_clusters = n_clusters
        self.max_iter = max_iter
        self.centroids = None

    def initialize_and_fit(self, initialization_method: InitializationMethod,
                           X_train: np.ndarray):
        # Initialize based on provided method choice.
        if initialization_method == InitializationMethod.FirstK:
            self.centroids = self.__initialize_first_k(X_train)
        elif initialization_method == InitializationMethod.Random:
            self.centroids = self.__initialize_random(X_train)
        elif initialization_method == InitializationMethod.MaxDistance:
            self.centroids = self.__initialize_max_distance(X_train)
        elif initialization_method == InitializationMethod.KMeansPlusPlus:
            self.centroids = self.__initialize_kmeanspp(X_train)

        # Some validation to be sure your methods are at least somewhat behavi
        assert not self.centroids is None  # The centroids were set.
        assert len(self.centroids
                   ) == self.n_clusters  # n_clusters of them were chosen.

        # Fit to the given centroids.
        num_iterations_needed = self.__fit(X_train)
        return num_iterations_needed

    def __initialize_first_k(self, X_train: np.ndarray):
        """
        Initialize the centroids by selecting the first k points in X_train.
        """
        centroids = []

        # YOUR CODE STARTS HERE
        centroids = X_train[:self.n_clusters]
        # YOUR CODE ENDS HERE

        return centroids

    def __initialize_random(self, X_train: np.ndarray):
        """
        Initialize the centroids by selecting k different points uniformly at
        random from X_train.

        (Hint: np.random.choice has a 'replace=False' argument to ensure
        points are unique.)
        """
        centroids = []
```

```
        # YOUR CODE STARTS HERE
        # https://www.geeksforgeeks.org/python/numpy-random-choice-in-python/
        centroids = X_train[np.random.choice(X_train.shape[0], size=self.n_clu
        # YOUR CODE ENDS HERE

        return centroids

    def __initialize_max_distance(self, X_train: np.ndarray):
        """
        Initialize the centroids iteratively to be as far apart as possible.

        1. Pick the first centroid uniformly at random from X_train.
        2. For each of the remaining k-1 centroids:
           a. For each data point, find its distance to the *nearest*
              centroid that has already been selected.
           b. Select the data point that has the *maximum* of these
              "nearest" distances as the new centroid.
        """
        # Pick a random point from train data for first centroid.
        centroids = [random.choice(X_train)]

        # Then choose the remaining points by selecting the point with maximum
        # euclidian distance to any current centroid.
        #
        # YOUR CODE STARTS HERE
        #step 2 (step 1 was given)
        for _ in range(1, self.n_clusters):
            nearest_dists = []
            for x in X_train:
                #step 2a
                all_dists = euclidean_dist(x, centroids)
                nearest_dists.append(min(all_dists))
            #step 2b
            max_dist_index = np.argmax(nearest_dists)
            centroids.append(X_train[max_dist_index])
        # YOUR CODE ENDS HERE

        return centroids

    def __initialize_kmeanspp(self, X_train: np.ndarray):
        """
        Initialize the centroids using the k-Means++ algorithm.

        1. Pick the first centroid uniformly at random from X_train.
        2. For each of the remaining k-1 centroids:
           a. For each data point x, calculate the squared distance to the
              *nearest* centroid that has already been selected.
              Let this be D(x)^2.
           b. Select the next data point from X_train at random, with a
              probability proportional to its D(x)^2 value.

              (Hint: np.random.choice has a 'p' argument for probabilities).
```

```
        """

        # Pick a random point from train data for first centroid.
        centroids = [random.choice(X_train)]

        # Then choose the remaining points as per kMeans++ algorithm.
        #
        # YOUR CODE STARTS HERE
        for _ in range(1, self.n_clusters):
            squared_dists = []
            for x in X_train:
                #easier to use numpy arrary for this operation
                centroids_nparray = np.array(centroids)
                all_sq_dists = np.sum((x - centroids_nparray) ** 2, axis=1)
                nearest_sq_dist = np.min(all_sq_dists)
                squared_dists.append(nearest_sq_dist)
            #step 2b
            d_sq = np.array(squared_dists) #D(x)^2 values described in 2a
            probabilities = d_sq / np.sum(d_sq)
            next_centroid = np.random.choice(len(X_train), size=1, p=probabili
            centroids.append(X_train[next_centroid[0]])
        # YOUR CODE ENDS HERE

        return centroids

    def __fit(self, X_train: np.ndarray):
        """
        Iterate, adjusting centroids until converged (new centroids are the sa
        as previous centroids) or until passed max_iter.

        Returns the number of iterations needed for the choice of centroids to
        stabilize.
        """
        iteration_count = 0
        prev_centroids = None

        while np.not_equal(self.centroids,
                prev_centroids).any() and iteration_count < self.max_iter:
            # Sort each datapoint, assigning to nearest centroid
            # Push current centroids to previous, reassign centroids as mean c
            #
            # YOUR CODE STARTS HERE
            prev_centroids = self.centroids
            # assignment
            cluster_assignments = np.zeros(len(X_train), dtype=int)
            for i, x in enumerate(X_train):
                dists = euclidean_dist(x, self.centroids)
                cluster_assignments[i] = np.argmin(dists)

            # update
            new_centroids = []
            for k in range(self.n_clusters):
                points_in_cluster = X_train[cluster_assignments == k]
```

```
                new_centroid = np.mean(points_in_cluster, axis=0)
                new_centroids.append(new_centroid)
            self.centroids = np.array(new_centroids)
            # YOUR CODE ENDS HERE
            iteration_count += 1

        return iteration_count

    def predict(self, X: np.ndarray):
        """
        Assign each data point to the nearest centroid.
        """
        centroids = []
        centroid_idxs = []
        for x in X:
            dists = euclidean_dist(x, self.centroids)
            centroid_idx = np.argmin(dists)
            centroids.append(self.centroids[centroid_idx])
            centroid_idxs.append(centroid_idx)

        return centroids, centroid_idxs
```

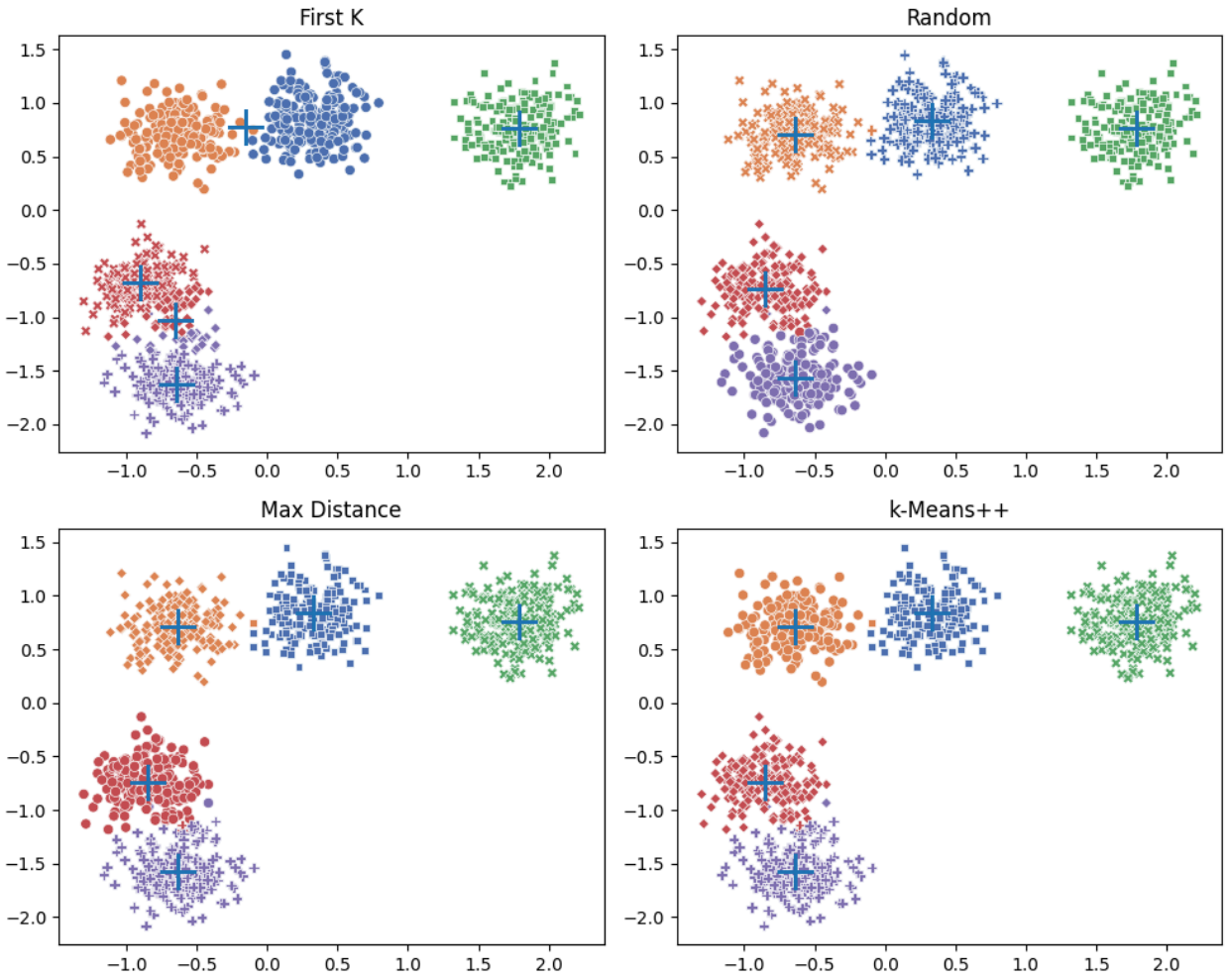
Now, **test and visualize your k-means algorithm** with various initializations. In the below given code, we separate the different true labels by color (as previously), and we distinguish predicted labels by marker styles.

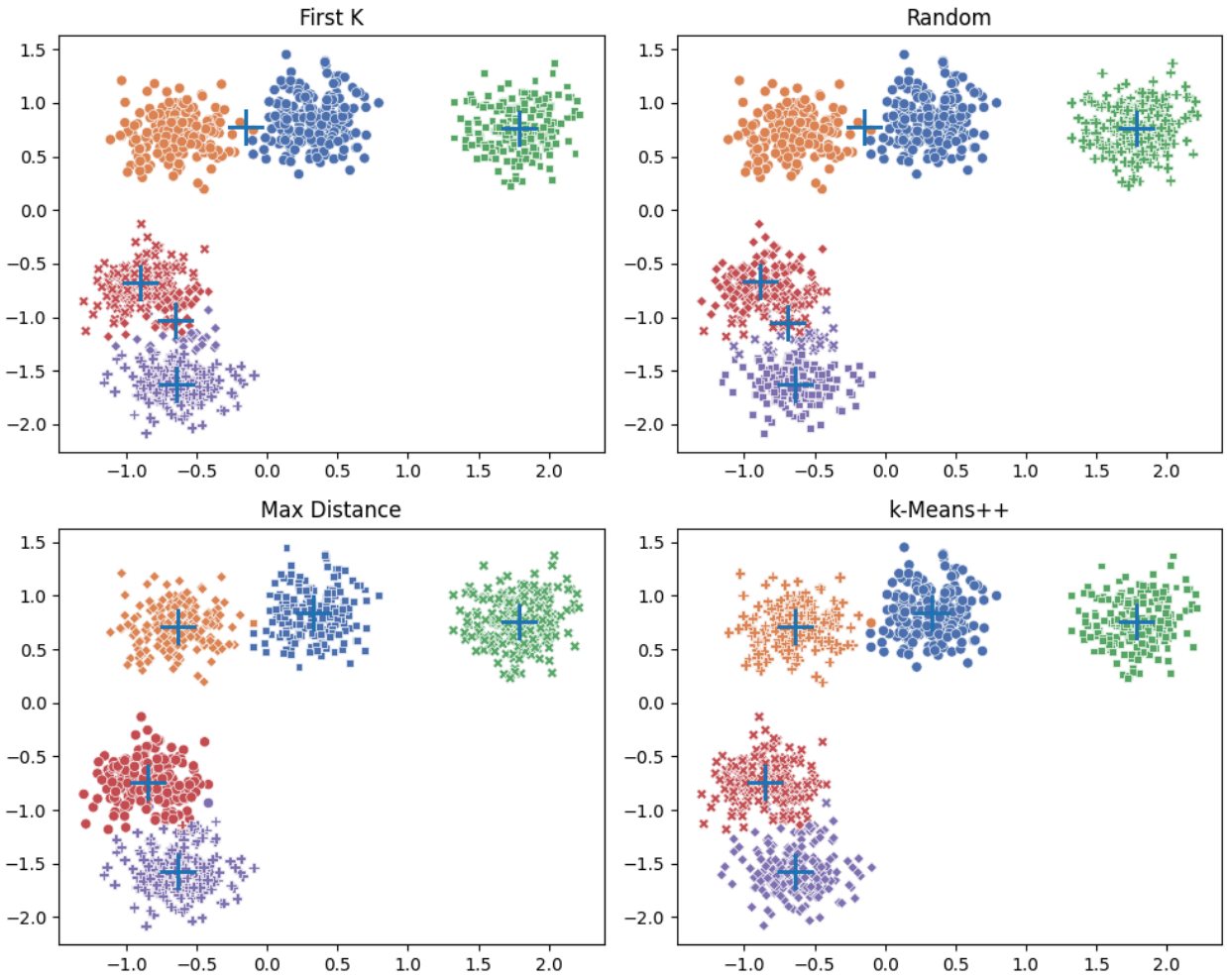
You should make sure all centroids ("+" signs) are in different clusters. Since initialization is important for k-means, so this may not be true for every run.

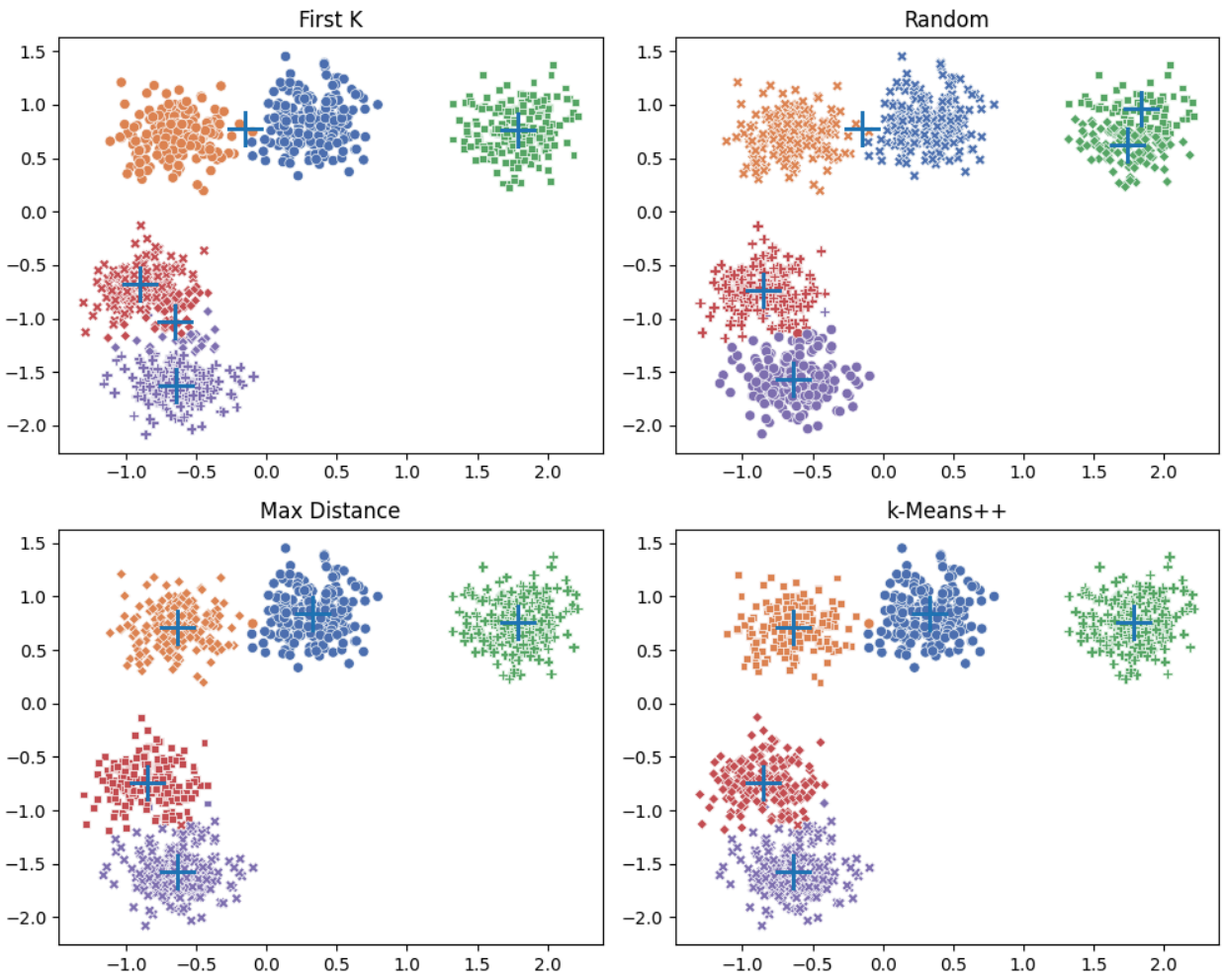
The below code will generate 3 charts, each containing 4 sub-plots. **Include these in your report, and comment on your observations**.

In [23]:
```
import matplotlib.pyplot as plt
import numpy as np

initializetion_methods = [
    (InitializationMethod.FirstK, "First K"),
    (InitializationMethod.Random, "Random"),
    (InitializationMethod.MaxDistance, "Max Distance"),
    (InitializationMethod.KMeansPlusPlus, "k-Means++")
]

def plot_predictions(model : KMeans, X : np.ndarray, y : np.ndarray):
    """Generates a single chart with subplots for multiple model prediction ru

    fig, axes = plt.subplots(2, 2, figsize=(10, 8))  # 2x2 grid of subplots
    axes = axes.flatten()  # Flatten the axes array for easier indexing

    for i in range(len(initializetion_methods)):
        method, name = initializetion_methods[i]
```

```
        # Initialize with given method.
        model.initialize_and_fit(method, X_train)

        # View results
        class_centers, classification = model.predict(X_train)

        # plot the ground truths
        sns.scatterplot(
            x=X,
            y=y,
            hue=true_labels,
            style=classification,
            palette="deep",
            legend=None,
            ax=axes[i]
        )
        # plot the centroids
        axes[i].plot(
            [x for x, _ in class_centers],
            [y for _, y in class_centers],
            '+',
            markersize=20,
        )

        axes[i].set_title(name)

    plt.tight_layout()  # Adjust subplot params for a tight layout
    plt.show()

# Run k-means with each initialization method.
model = KMeans(n_clusters=num_centers)
for _ in range(3):
    plot_predictions(model, [X[0] for X in X_train], [X[1] for X in X_train])
```


First K
Random
Max Distance
k-Means++
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−2.0
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0


First K
Random
Max Distance
k-Means++
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−2.0
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0



Observations:

One of my first observations is that First-K and Random do not have consistent clustering results. First K failed to define each separately colored cluster and Random init only had one trial where it succeeded in determining the right cluster groups. This likely means that these algorithms are too sensitive to the initialization of the first centroid. If the first centroid is not ideally picked, then the clustering will not necessarily be successful as it can be.

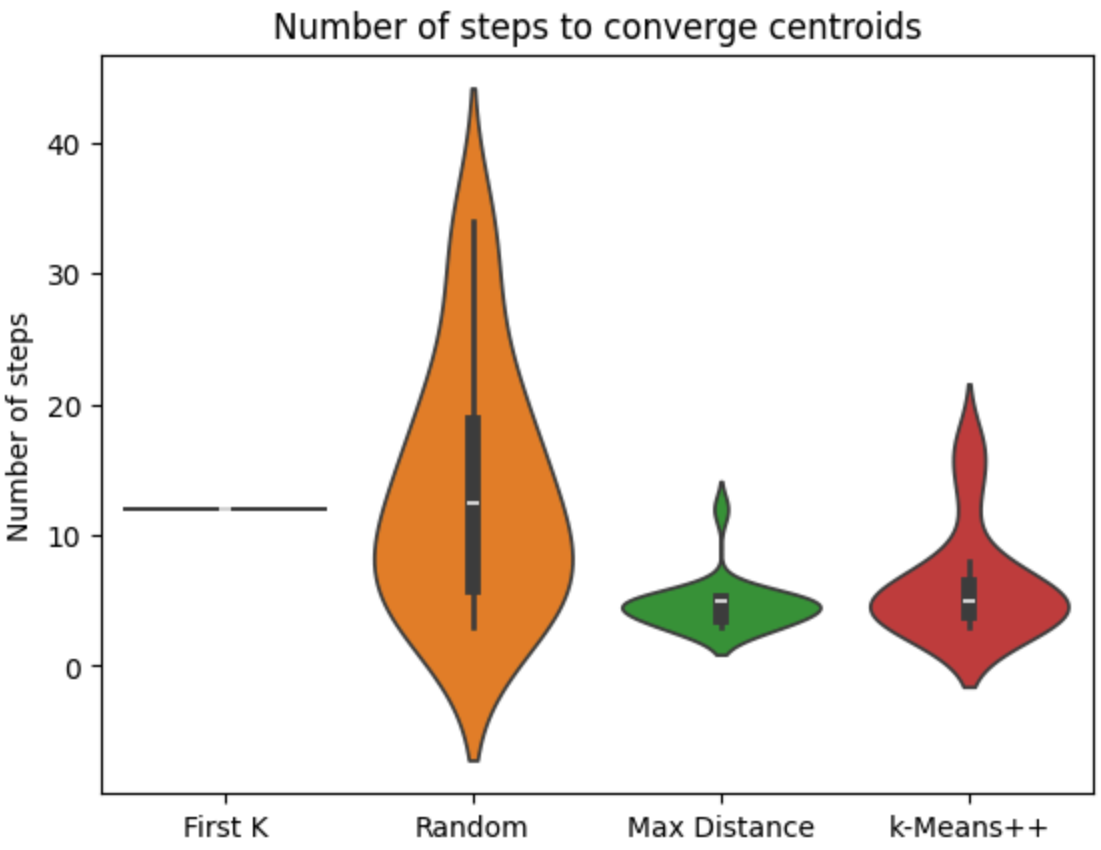
K-means++ and Max-Distance did a great job every trial.

The accuracy of these different methods can be seen above, but that's only the *end result*! As you can see in your code, each of these initialization methods is of varying complexity.

From each initialization state, how many steps does it take to complete initialization? **Include this graph in your report, and comment on the results.**

Personal Note: This step was taking way too long to run so I had to go up and check my code.

In [24]:
```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

iterations = 20

# Run k-means with each initialization method.
model = KMeans(n_clusters=num_centers)

data = []
for i in range(len(initializetion_methods)):
    method, name = initializetion_methods[i]

    num_steps = []
    for i in range(iterations):

        # Initialize with given method.
        num_steps.append(model.initialize_and_fit(method, X_train))

    data.append((num_steps, name))

sns.violinplot([d[0] for d in data])
plt.xticks(np.arange(len(data)), labels=[d[1] for d in data])
plt.title("Number of steps to converge centroids")
plt.ylabel("Number of steps")
plt.show()
```



Observations:

There is huge variation in convergence betweek First-K and Random while max-idstance and k-means++ have more comparable results for the number of steps it takes to converge centroids. It looks like first-k converges in the same amount of steps every time. Random has a much wider distribution and is more variable in how long it takes to converge. This makes some sense considedring the consistency of a bad result for k-means and the fact that Random was able to successfully assign centroids 1/3 times - more variation in it's run means there's maybe a higher chance of it giving a good result than k-means which is consistent in it's results.

Both max-distance and k-means++ start off with a better set of intitialized centroids and therefore have less work to do to reach convergence -- less steps. So, overall the distribution and the average of the # of steps of convergence is lower than both first-k and random.

Explanation of the Code

Explain how your code in each of the above sections works, using a couple of sentences per function to describe your code structure and any choices you made

in its implementation.

euclidean_dist: This just calculates the standard euclidean distance which is the square root of the sum of differences squared between a single cur_point and all other points in the dataset. It then returns a list of distances between a single point and every other point. Useful helper to get all distances to other points, for a single point, in our other functions.

__initialize_first_k: This selects the first k data points from the X_train dataset as initial centroids as per the instructions in the doc string of the function.

__initialize_random: Selects k unique, random indices from X_train (using the np.random.choice hint) and uses those points as initial centroids.

__initialize_max_distance: First, this function picks a random centroid. Now that one centroid has been chosen, we need to find the other k-1 centroids. So we look for the point in x-train the furthest away from the first centroid. WE then add the new centroid to the list and repeat this process, and make sure the next point is far away from all existing centroids.

__initialize_kmeanspp: Here we pick one random centroid and then for the k-1 remaining centroids we calculate the squared distance for every point to its nearest existing centroid. Specifically, all_sq_dists = np.sum((x - centroids_nparray) ** 2, axis=1): For a given point x, calculates the squared distance to all the centroids that have already been chosen. Then we select the next centroid by calculating probabilities (numerator is one squared distance over the sum of all squared distances). The chance of a point being chosen is proportional to its squared distance in virtue of the probability I just described, which allows this method to have a similar advantage to max-distance -- picking points farther from existing centroids.

__fit: this is just the k-means algorithm. WE loop until the centroids stop changing or the max_iter parameter value is reached. There are two steps: assignment and update. For the assignment, each point in X_train is assigned to the cluster of its nearest centroid (using our euclidean_dist helper). Then, for the update, the

note, i used grammarly to fix my spelling errors since spelling errors do not show up in VS Code.

Questions

1. In practice, both k-Means++ and Max Distance are used. In what conditions might you prefer one of these methods over another? What are the drawbacks of each method?

2. The project told you to use $k=5$, but in the real world, k is unknown. A common heuristic is the 'Elbow Method,' which plots the Sum of Squared Errors (SSE) for different values of k. First, describe what you would expect this plot to look like (e.g., how does SSE change as k increases from 1 to n?). Then, explain why this plot is used to find an 'elbow' and what this 'elbow' point conceptually represents.

3. Your violin plot above shows the number of iterations to converge, but this isn't the full story. In Big-O notation (in terms of n data points, m features, and k clusters), what is the time complexity of the `__initialize_random method` versus the `__initialize_max_distance` method? Given this, explain the performance trade-off you are making when you choose one over the other?

Answers

Q1: You would generally prefer k-Means++ since it is probabilistic approach (as we saw in class, making a method probabilistic usally makes the non-probabilistic version of the algorithm better) and more robust to outliers and often leads to better final clustering results. Still, if you need a highly reproducible result, you can use max-distance since it's more of a deterministic method for initialization. You should make sure that the data doesn't have any outliers though.

Drawbacks: k-Means++'s main drawback is that it is non-deterministic because it has a random component. So, running the initialization multiple times can produce slightly different starting centroids, which may lead to slightly different final clusterings on the same data.

Max Distance's main drawback is it's sensitivity to outliers. If there is one point far from all other data (an outlier) it is going to be chosen as the centroid, which can set off a chain reaction of weird centroid placements.

Q2: I would expect this plot to look decreasing because when k=1, all points would be in a single cluster and the SSE would likely be at its max since the data need not be close together to be in a single cluster since k = 1. Then, As k increases the error would decrease because points will be closer to their respective centroids (since there are more). Then finally, when k becomes the number of points, n, every point is just its own centroid, and the error becomes 0 (this is overfit)

This plot can be used to find the elbow which visualy represent a bend in the plot that represents where the rate of SSE decrease flattens out. This "elbow" gives us the optimal k for the algorithm because it represent the point at which the model doest not reduce error in a signifcant enough way. So, this point helps us from picking too high of a k (overfitting) and too small of a k (makign sure our sse is reduced enough) - a balance between model complexity and low error.

Q3: __initialize_random: The complexity is O(dot_product(k, m)) to copy the k points, each with m amount of features. This doesn't depend on n at all.

__initialize_max_distance: This func does $k-1$ iterations but within each iteration it computes the distance from all n points to all existing centroids. This step is $O(n \cdot i \cdot m)$. The total complexity is the sum of these steps throughout every iteration(k times) so the complexity comes out to $O(k^2 \cdot n \cdot m)$.

So the tradeoff is that the Random method costs less to initialize ($O(k \cdot m)$) but also is more variable in it's results and tendency to create a good clustering result. This can lead to the fit algorithm requiring a lot of iterations to converge. There is also a higher risk of getting a worse final SSE that gets stuck in a bad local optimum. On the other hand, Max Distance has a very high init cost but you get a much better initialization of centroids which can lead to faster convergence and less interations in the main fit function (plus an overall better clustering result).

Task 2: Principal Component Analysis (50 points)

In this problem you will apply PCA for dimensionality reduction to the MNIST dataset.

Note that **you must implement PCA directly with Numpy**, you are not allowed to use the PCA class from scikit-learn or any other PCA/dimensionality reduction

library.

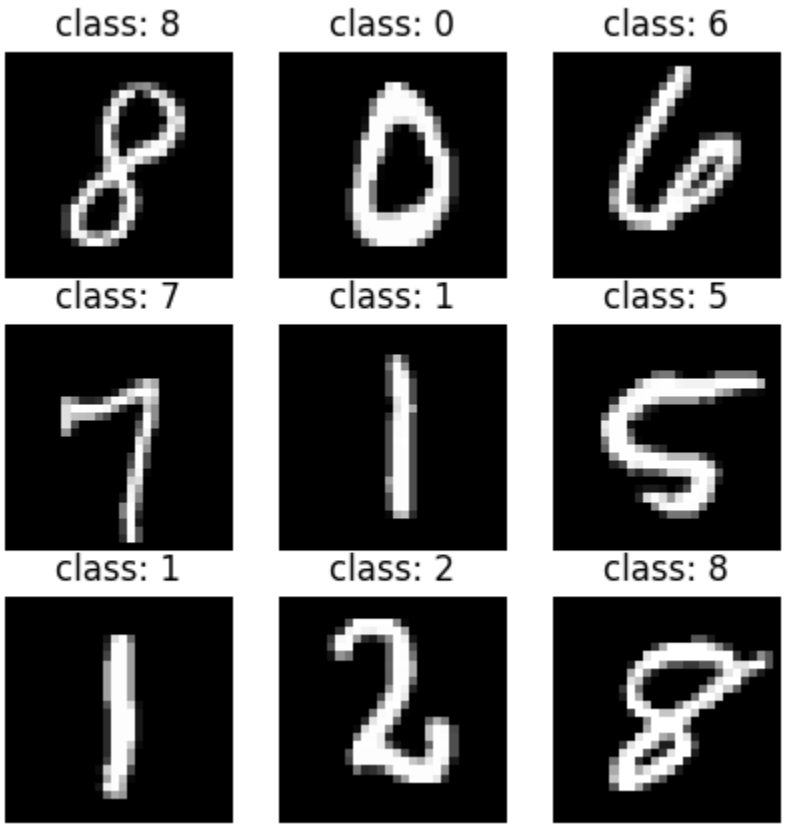
Load the dataset and visualize some examples

In [25]:
```
# Download dataset
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split

print('Loading and preprocessing data')
# Load data from https://www.openml.org/d/554
X, y = fetch_openml("mnist_784", version=1, return_X_y=True, as_frame=False, c
y = y.astype(int)

X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0, test
```

```
Loading and preprocessing data
```

```
/Users/rheamadhogarhia/Library/Python/3.9/lib/python/site-packages/sklearn/data
sets/_openml.py:72: RuntimeWarning: Invalid cache, redownloading file
  warn("Invalid cache, redownloading file", RuntimeWarning)
```

In [26]:
```
# plot some random samples from training set
import matplotlib.pyplot as plt
fig, axes = plt.subplots(3, 3, figsize=(5, 5))
for i, ax in enumerate(axes.flat):
    ax.imshow(X_train[i].reshape(28, 28), cmap='gray')
    ax.set_title(f'class: {y_train[i]}')
    ax.axis('off')
```



A simple linear classifier

This class has been implemented for you. You don't need to change code here.

In [27]:
```python
# Don't Change this cell
class LinearClassifier:
    """
    Simple linear classifier trained using full-batch gradient descent.
    - Maps n_features -> n_classes logits using a weight matrix W and bias vec
    - Applies softmax to the logits to get class probabilities that sum to 1.
    - In prediction, we take the class with the highest probability as our out
    """
    def __init__(self, n_features, classes=np.arange(10), eps=1e-4):
        self.classes = classes
        self.W = np.random.rand(n_features, len(classes)) / 30
        self.b = np.random.rand(len(classes)) / 30
        self.eps = eps

    def fit(self, X, y):
        assert len(X) == len(y)
        assert set(y) == set(self.classes)

        for i in range(200):
            # forward pass
            z = X @ self.W + self.b

            # softmax to convert logits to class probabilities
            exp_z = np.exp(z)
            softmax = exp_z / exp_z.sum(axis=1, keepdims=True)

            # cross-entropy loss
            n = len(X)
            loss = -np.log(softmax[range(n), y]).sum() / n

            # backward pass
            dL_dz = softmax
            dL_dz[range(n), y] -= 1
            dL_dz /= n

            dL_dW = X.T @ dL_dz
            dL_db = dL_dz.sum(axis=0)

            # update weights
            self.W -= self.eps * dL_dW
            self.b -= self.eps * dL_db

            # print classification accuracy
            if i % 50 == 0:
                predictions = np.argmax(z, axis=1)
                accuracy = (predictions == y).mean()
                print(f'iteration {i}: loss {loss:.4f}, accuracy {accuracy:.4f
        predictions = np.argmax(z, axis=1)
```

```
        accuracy = (predictions == y).mean()
        print(f'Final iteration {i}: loss {loss:.4f}, accuracy {accuracy:.4f}'

    def predict(self, X):
        z = X @ self.W + self.b
        return np.argmax(z, axis=1)
```

Baseline

As a baseline, let's report the accuracy of a simple linear classifier trained with full-batch gradient descent. Instead of using all 28 x 28 = 784 pixels as input, we'll uniformly sample 27 pixels from the image and see what our accuracy is.

You should see something around ~57% train accuracy and ~57% test accuracy.

In [28]:
```
# subsample every 30 pixels
X_train_subset = X_train[:, ::30]
print(f'Using {X_train_subset.shape[1]} features')

classifier = LinearClassifier(n_features=X_train_subset.shape[1])

# center the data and store the means
feat_means = X_train_subset.mean(axis=0, keepdims=True)
X_train_subset = X_train_subset - feat_means
classifier.fit(X_train_subset, y_train)
```

```
Using 27 features
iteration 0: loss 6.1824, accuracy 0.0883
/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/ipykernel_26458/3174790117.p
y:21: RuntimeWarning: divide by zero encountered in matmul
  z = X @ self.W + self.b
/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/ipykernel_26458/3174790117.p
y:21: RuntimeWarning: overflow encountered in matmul
  z = X @ self.W + self.b
/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/ipykernel_26458/3174790117.p
y:21: RuntimeWarning: invalid value encountered in matmul
  z = X @ self.W + self.b
/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/ipykernel_26458/3174790117.p
y:36: RuntimeWarning: divide by zero encountered in matmul
  dL_dW = X.T @ dL_dz
/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/ipykernel_26458/3174790117.p
y:36: RuntimeWarning: overflow encountered in matmul
  dL_dW = X.T @ dL_dz
/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/ipykernel_26458/3174790117.p
y:36: RuntimeWarning: invalid value encountered in matmul
  dL_dW = X.T @ dL_dz
iteration 50: loss 1.4437, accuracy 0.5343
iteration 100: loss 1.3588, accuracy 0.5660
iteration 150: loss 1.3383, accuracy 0.5729
Final iteration 199: loss 1.3294, accuracy 0.5750
```

sanity check! Yes, i do see around 57% accuracy

In [29]:
```
# evaluate our test accuracy
X_test_subset = X_test[:, ::30]
preds = classifier.predict(X_test_subset - feat_means)
test_acc = (preds == y_test).mean()
print(f'Test accuracy: {test_acc:.4f}')
```

```
Test accuracy: 0.5746
```

```
/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/ipykernel_26458/3174790117.p
y:53: RuntimeWarning: divide by zero encountered in matmul
  z = X @ self.W + self.b
/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/ipykernel_26458/3174790117.p
y:53: RuntimeWarning: overflow encountered in matmul
  z = X @ self.W + self.b
/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/ipykernel_26458/3174790117.p
y:53: RuntimeWarning: invalid value encountered in matmul
  z = X @ self.W + self.b
```

PCA for dimensionality reduction in classification

Can we use PCA to choose a more effective set of features?

Following the starter code below to implement PCA:

- First, **implement the fit function** which takes the data as input, and compute and store the truncated SVD into self.U_subset, self.S_subset, self.Vt_subset.
- Then, **implement the predict function**, which projects the data onto our kept principal components.

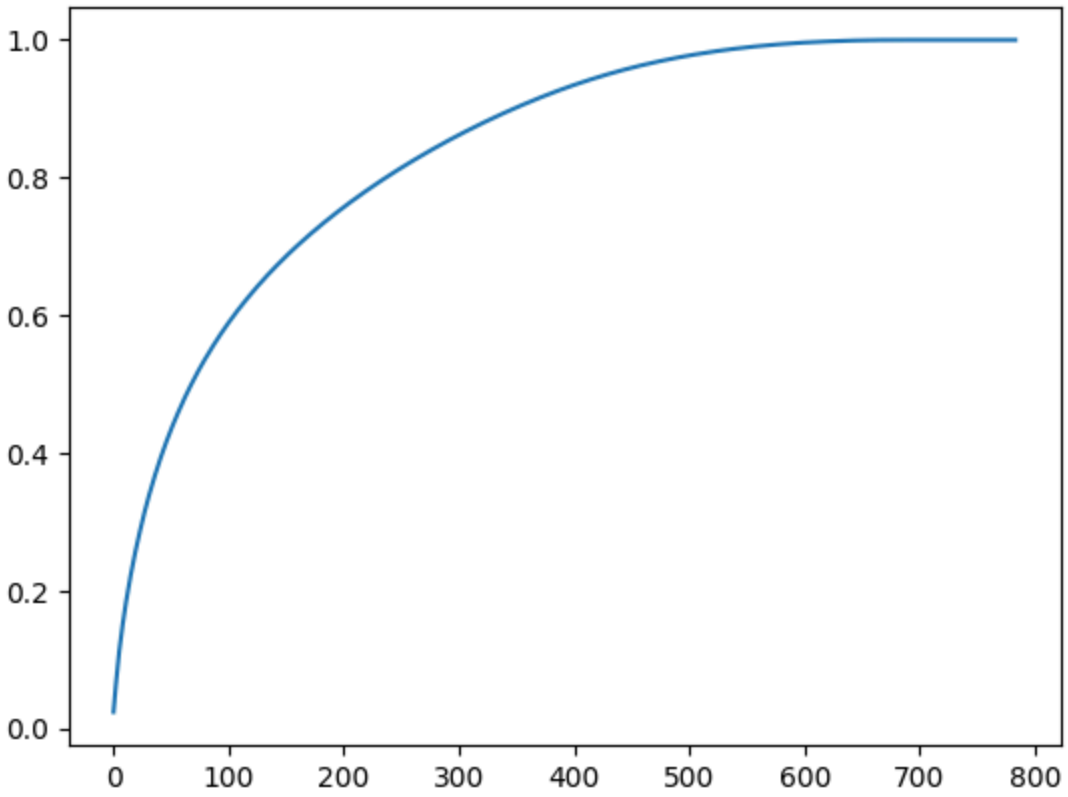
In [30]:
```
class PCA:
    def __init__(self, n_components):
        # number of principal components to keep when we apply PCA
        assert n_components <= 784
        self.n_components = n_components

        self.feature_means = None
        self.U, self.S, self.Vt = None, None, None
        self.U_subset, self.S_subset, self.Vt_subset = None, None, None

    def fit(self, X):
        """
        Steps:
        - Center the data (compute, store, and subtract the mean of each featu
        - Take the SVD of the centered data
        - Truncate the SVD by keeping only the first n_components (singular va
        """
        n_samples, n_features = X.shape
```

```
        ## Your Code Starts Here ##
        # center the data per feature
        self.feature_means = np.mean(X, axis=0)
        centered_X = X - self.feature_means
        # take SVD
        self.U, self.S, self.Vt = np.linalg.svd(centered_X, full_matrices=Fals
        # store truncated SVD
        self.Vt_subset = self.Vt[:self.n_components, :]
        self.U_subset = self.U[:, :self.n_components]
        self.S_subset = self.S[:self.n_components]
        ## End of Your Code ##

    def predict(self, X):
        """
        Steps:
        - Center the data (subtract the mean of each feature, stored in the fi
        - Project the centered data onto the principal components

        Returns:
        - Projected data of shape (X.shape[0], n_components)
        """
        if self.feature_means is None:
            raise ValueError('fit the PCA model first')

        ## Your Code Starts Here ##
        # center X
        centered_X = X - self.feature_means
        # project the centered data set onto our kept principal components
        projected_X = centered_X @ self.Vt_subset.T
        return projected_X
        ## End of Your Code ##
```

In [31]:
```
# fit PCA
pca = PCA(27)
pca.fit(X_train)
```

In [32]:
```
# plot the explained variance ratio
S = pca.S
plt.plot(S.cumsum() / S.sum())

# report the truncated feature ratio
explained_var_ratio = (S.cumsum() / S.sum())[pca.n_components]
print(f'Explained variance ratio with {pca.n_components} components: {explaine
```

```
Explained variance ratio with 27 components: 0.3143
```



In [33]:
```
X_train_subset = pca.predict(X_train)
X_test_subset = pca.predict(X_test)

classifier = LinearClassifier(n_features=27)
classifier.fit(X_train_subset, y_train)
```

```
iteration 0: loss 19.0710, accuracy 0.1311
```

```
/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/ipykernel_26458/1370318126.p
y:48: RuntimeWarning: divide by zero encountered in matmul
  projected_X = centered_X @ self.Vt_subset.T
/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/ipykernel_26458/1370318126.p
y:48: RuntimeWarning: overflow encountered in matmul
  projected_X = centered_X @ self.Vt_subset.T
/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/ipykernel_26458/1370318126.p
y:48: RuntimeWarning: invalid value encountered in matmul
  projected_X = centered_X @ self.Vt_subset.T
/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/ipykernel_26458/3174790117.p
y:21: RuntimeWarning: divide by zero encountered in matmul
  z = X @ self.W + self.b
/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/ipykernel_26458/3174790117.p
y:21: RuntimeWarning: overflow encountered in matmul
  z = X @ self.W + self.b
/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/ipykernel_26458/3174790117.p
y:21: RuntimeWarning: invalid value encountered in matmul
  z = X @ self.W + self.b
/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/ipykernel_26458/3174790117.p
y:36: RuntimeWarning: divide by zero encountered in matmul
  dL_dW = X.T @ dL_dz
/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/ipykernel_26458/3174790117.p
y:36: RuntimeWarning: overflow encountered in matmul
  dL_dW = X.T @ dL_dz
/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/ipykernel_26458/3174790117.p
y:36: RuntimeWarning: invalid value encountered in matmul
  dL_dW = X.T @ dL_dz
iteration 50: loss 0.6588, accuracy 0.8613
iteration 100: loss 0.4626, accuracy 0.8766
iteration 150: loss 0.4195, accuracy 0.8787
Final iteration 199: loss 0.4139, accuracy 0.8793
```

Evaluate our test accuracy with PCA. You are expected to achieve a higher accuracy ~80%.

In [34]:
```
preds = classifier.predict(X_test_subset)
test_acc = (preds == y_test).mean()
print(f'Test accuracy: {test_acc:.4f}')
```

```
Test accuracy: 0.8789
/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/ipykernel_26458/3174790117.p
y:53: RuntimeWarning: divide by zero encountered in matmul
  z = X @ self.W + self.b
/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/ipykernel_26458/3174790117.p
y:53: RuntimeWarning: overflow encountered in matmul
  z = X @ self.W + self.b
/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/ipykernel_26458/3174790117.p
y:53: RuntimeWarning: invalid value encountered in matmul
  z = X @ self.W + self.b
```

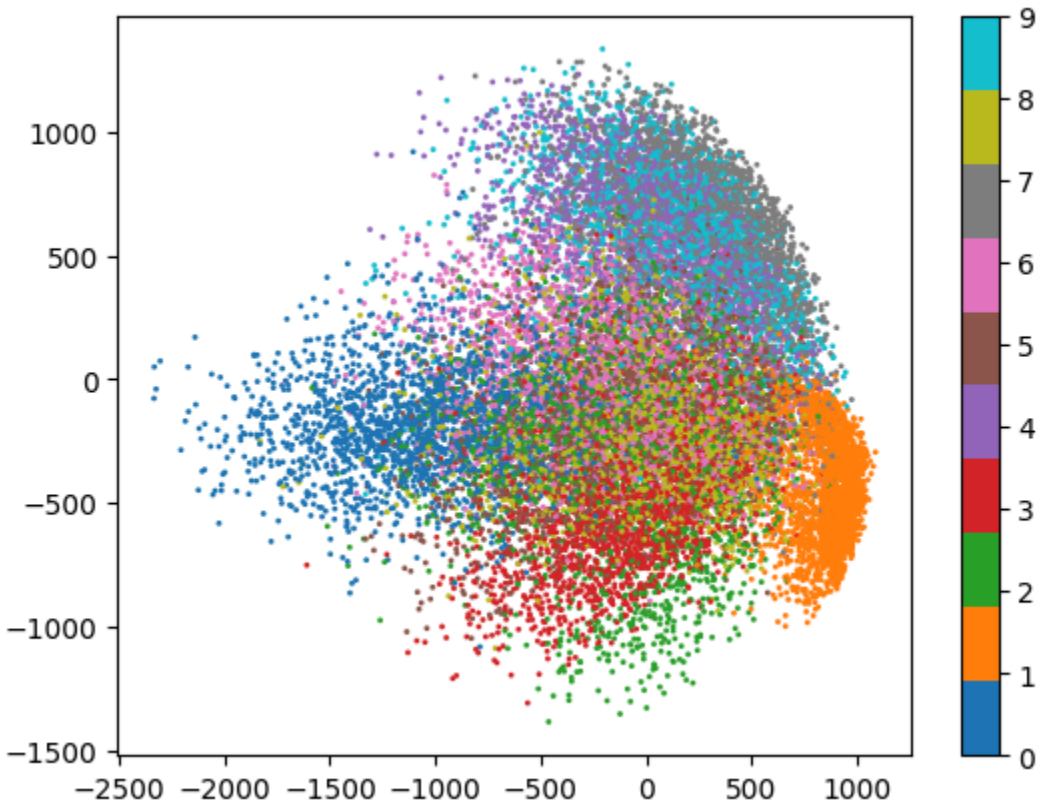
PCA for visualization

The following block of code will visualize the first 2 principal components. You should discuss what you see by comparing the clusters of different digits in the 2D space.

In [35]:
```
# extract the first two PCs from the training set and plot it
X_train_subset = pca.predict(X_train)
X_train_subset = X_train_subset[:, :2]

plt.scatter(X_train_subset[:, 0], X_train_subset[:, 1], c=y_train, cmap='tab10
plt.colorbar()
```

```
/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/ipykernel_26458/1370318126.p
y:48: RuntimeWarning: divide by zero encountered in matmul
  projected_X = centered_X @ self.Vt_subset.T
/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/ipykernel_26458/1370318126.p
y:48: RuntimeWarning: overflow encountered in matmul
  projected_X = centered_X @ self.Vt_subset.T
/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/ipykernel_26458/1370318126.p
y:48: RuntimeWarning: invalid value encountered in matmul
  projected_X = centered_X @ self.Vt_subset.T
```

Out[35]: <matplotlib.colorbar.Colorbar at 0x1588d6610>



Observations: This 2d plot represents the data projected on two the reduced dimensions (the first two principal components). It makes sense that 0 and 1's

clusters are further apart form one another because their visual shapes (an oval and a line) are pretty distinct from one another.

This classifier is not great at separating 4's from 9s or 3's from 8's since these both have similar structures to one another (a 3 is like half of an 8).

Explanation of the Code

Explain how your code in each of the above sections works, using a couple of sentences per function to describe your code structure and any choices you made in its implementation.

fit: for the pca fit function I center the data by calculating the mean of each pixel and then subtracting it from all data points. Basically, as discussed in class, this allows PCA to find variance in teh dta's structure, not just the average position. I then use numpy's svd solver to perform SVD on the data (the centered data). I then store the first n principal compoents of the matrix (the eigenvectors which capture the most vairance in the data). n_components is defined in the given __init

predict: I use the stored feature_means from fit to center the data again and then I project out centered data set onto the n number of principal components defined in the init function. I do this through matrix multiplication (which in python I can do with @). This multiplication also uses the stored V transpose subset we got from thefit function. This will result in us transforming the original dataset into a n-compnent feature space!

The rest of the code was given (for plotting and visualization).

Questions

1. Your baseline classifier used 27 randomly selected pixels and achieved ~57% accuracy. Your PCA-based classifier used 27 principal components and achieved ~80% accuracy. Explain conceptually *why* the 27 principal components are so much more "informative" than the 27 random pixels. What does a single principal component represent (in terms of the original pixels) that a single random pixel does not?

2. Look at your 2D scatter plot of the first two principal components. Based on this plot, which two or three digits (e.g., '1' and '7', '4' and '9') appear to be the most difficult to separate from each other? Which digit appears to form the most distinct and well-separated cluster? Briefly explain why this makes intuitive sense, based on the physical

appearance (i.e., how the digits are written) of the digits you identified.

3. In both your fit and predict methods, a required step is to "center the data" by subtracting the feature means. What is the conceptual reason for this? What might go wrong if you performed PCA on the *un-centered* data?

Answers

Q1: Pixels and Principal components are fundamentally very different things. A pixel is a single piece of information about the overall dataset and carries little information about the dataset besides itself. It's just a small sample of the image... a pixel. But, a principal component is a linear combo of all of the pixels in the image and is weighted to capture not just the data that those pixels represent, but also the relationships between the pixels - the variance and dimensional relations. The principal components can represent a direction or pattern about the dataset in a vectorized form. The first 27 principal components are the 27 most important patterns hidden in the data. WE can use these patterns to differentiate images - whether or not another image has the same most important patterns is important to classifcation. TLDR, the principal components carry info about the whole dataset whereas 27 random pixels only contain info about those 27 points.

Q2: I mentioned this in my observations above but 0 and 1 seem to be the easiest to distingish and (3,8) (5,6), and (4,9) also all seems to be hard to distinguish pairs. This makes intuitive sense because 0 and 1 have very different visual structures, one of them is circular, very oval, and the other is a straight line, a more rigid number. 1 is pretty skinny where 0 can be pretty wide. 3 and 8 makes sense because 3, especially if the ends of the 3 are extended a little more, could look like half or more of an 8, and 5 and 6, differ very minimally by closing the bottom left of the 5. Finally, 4 and 9 both have their closed regions facing the same way in regard to the open part of their tails. Thus, their visual and structural patterns all have similarities that make it hard (sometimes even for humans) to tell apart.

Q3: As I mentioned above in the code explanation, we center the data so that we can accurately capture variance in the data (which PCA aims to do - finding directions of maximum variance in the data). If we don't centere the data, we run the risk of finding variance as it relates to the magnitude of each data points position or values rather than the actual strucuter and spread of the data itself. Basically, we are scrubbing the importance of where the data lies numerically on a graph and trying to look at the positional ans structural components of the data. TLDR, we want to get rid of location bias and instead just capture patterns able to

found in the "shape" of the data.